cw1 Averagl

Machine Learning for Computer Vision

Coursework on Randomised Decision Forest

Shu Jiang
01225056
Department of Electrical and Electronic Engineering
sj4015@ic.ac.uk

Hao Xiong
01220667
Department of Electrical and Electronic Engineering
HX3115@ic.ac.uk

Abstract

Random forest is an ensemble of randomly trained decision trees which is a learning method for classification, regression and other tasks.

I INTRODUCTION

In this coursework, we mainly provide the simulation in MATLAB for observing the performance of Random Forest classifier under different parameters including tree number, tree depth, randomness parameter ρ and split function (weak learner). The concept of bag of visual words is also introduced and two common methods to construct codebook (K-means codebook and RF codebook) are applied in RF classifier to compare their performance.

II PROCEDURE

2.1 Training Decision Forest

In this section, 4 data subsets are generated from the training data by bagging. Bagging is implemented by sampling from the training data uniformly and with replacement and the sampling fraction is set as 100%. Each subset has 150 elements. The result is shown as Figure 1.

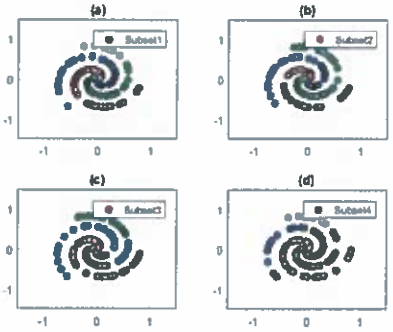


Figure 1 four subsets sampled from training data

Table 1 Unique data ratio of subsets

Subset	Theoretic	Subset 1	Subset 2	Subset 3	Subset 4
Ratio	0.623	0.667	0.613	0.653	0.640

Table 1 illustrates the unique data ratio of 4 subsets, it is obvious that when the subset size (n_i) equals the size of training data set (n), the practical value is close to the theoretical value. In other words, some data are repeated and each subset has 63.2% of unique data.

Then one of the data subsets is taken to grow a tree, 4 split functions are tried to split the node, they are axis-aligned, linear, non-linear and two-pixel node weak learner. Figure 2 shows the split functions and class histograms of the first node and its two children nodes respectively under different degree of randomness parameters.

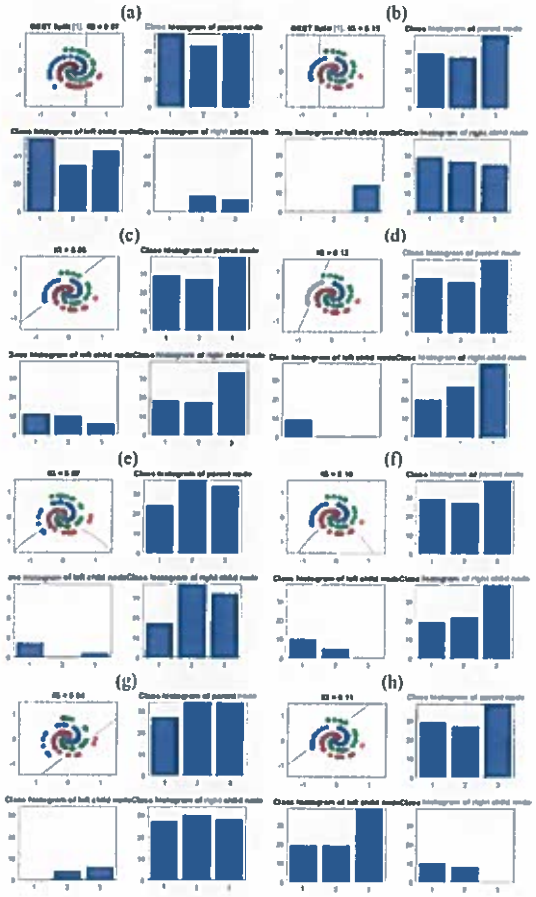


Figure 2 Four split functions and class histogram of first node and its children nodes: (a), (c), (e) and (g) are the results of axis-aligned, linear, nonlinear and two-pixel respectively when randomness parameter is set as 5, (b), (d), (f) and (h) are those of four split functions when randomness parameter equals 10.

The red line in the first image of each figure shows the split function. In axis-aligned, the geometric primitive used to separate the data is a line aligned to one of the axes; In linear weak learner, it is a generic line; In non-linear weak learner, it is a conic section. While in two-pixel test, one more dimension than that in axis-aligned

Summary of Comments on mlcv_cw1_average.pdf

Page: 1

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:56:32
excellent!

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:56:00

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:56:03
good

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:56:10

is used, the different signs of the two terms act like a discrete operator. From the second image of each figure, compared to the parent node, the children node 1stributions are purer, so their entropy is lower and their information gain is higher than their parent node.

Table 2 The best information gain of each split function

	axis-aligned	linear	nonlinear	two-pixel
ρ=5	0.07	0.03	0.07	0.04
ρ=10	3.15	0.12	0.10	0.11

From Table 2, it is obvious that the information gain (IG) increases when ρ grows which means the performance of splitting can be improved by increasing the degree of randomness in a tree. 4aximizing the information gain helps to select the split parameters which improves confidence in the final distributions. The result illustrates that the performance of four split functions are similar.

Then the nodes are spitted recursively and the tree growth is done when 7e maximum number of depth has been reached or 8 node contains too little training data (In this experiment, the threshold is set as 5), which is also known as stopping criteria. The final class distributions of some leaf nodes using four splitting functions are visualised as Figure 3.The reason for the selection of this stopping criteria is 10t growing full trees may have negative effects in terms of generalization. By stopping the growth of various branches of the tree when the stopping criteria is satisfied, we can control the shape of the tree.

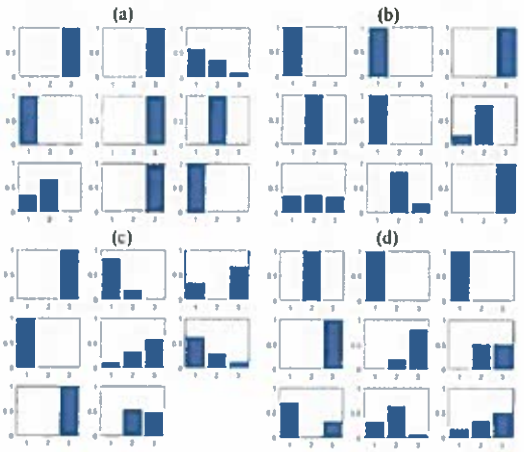


Figure 3 (a), (b), (c) and (d) are the class distribution of some leaf nodes of the first decision tree by implementing axis-aligned, linear, non-linear and two-pixel weak learner respectively.

2.2 Evaluating Decision Forest on the test data

In this section, novel data points in the test dataset are used to evaluate the random forest trained above. Figure 4 is the result of evaluation of the 4 given data points (-0.5,-0.7), (0.4, 0.3), (-0.7, 0.4) and (0.5,-0.5) in the test dataset.

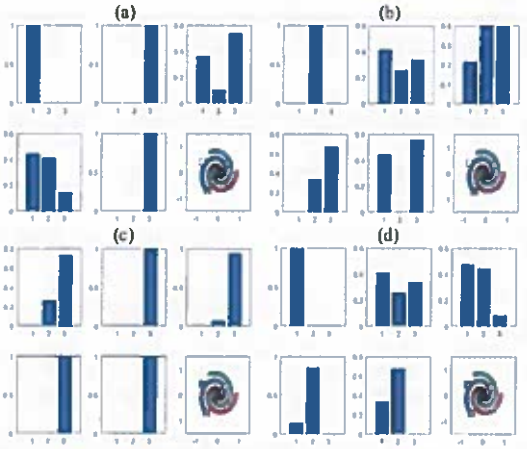


Figure 4 (a), (b), (c) and (d) are the evaluation of trained forest using four grabbed points in the test dataset one by one. In Figure 4 (a), the first two rows are the class distributions of the 6f nodes which the first test data point arrives at, while in the third row, the first image is the averaged class distribution and the second image is the visualisation of test result, which is the same in (b), (c) and (d).

In the last image of Figure 4(a), it is clear that four square spots with different colours represents the testing result of four given points. It is clear that the first point is located between two different class represented by blue and red, so the colour of this point is purple which is similar in (b), (c) and (d).

Then all the data points in the test dataset are evaluated and different parameter (the number of trees, the depth of trees and the degree of randomness) values of RF are tried to see their effects.

To see the effect of tree number, we fix the tree depth= 5, the degree of randomness ρ=3 and change the number of trees within {1, 4, 20, 50}.

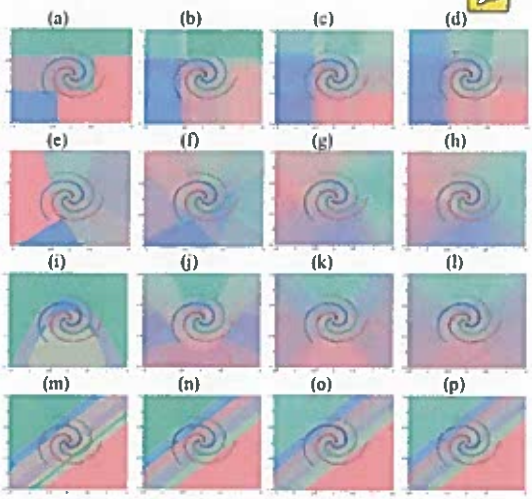


Figure 5 Show the effect of tree number ranging from {1, 4, 20, 50}. (a), (b), (c) and (d) are the classification of test data using axis-aligned weak learner when tree number equals 1, 4, 20

Page: 2

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:56:57

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:57:31
excellent, why the best information gain is attained with the axis-aligned?

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:57:08

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:57:43

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:57:47
very good

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:39:31

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:58:06

Number: 8 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:58:10

Number: 9 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:58:22
average, or majority voting?

Number: 10 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:58:23

Number: 11 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:40:21
excellent, very interesting combinatorial experiments

Number: 12 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:59:50
Do we earn higher information gain when using linear ones than axis-aligned?
Do we discuss different options of stopping?

and 50 respectively; (e), (f), (g) and (h) are the results of linear weak learner; (i), (j), (k) and (i) are those of non-linear weak leaner; (m), (n), (o) and (p) are those of two-pixel.

From the first column, it is obvious that when there is only one tree, the performance of classification is not so good for the overfitting problem. When the tree number grows from 4 to 50, the splitting result becomes better. At the same time, comparing four split function, it is clear that two-pixel weak learner is not suitable for this 2D dataset because in the two-pixel algorithm, there are two randomly selected latitudes which are equal to the dimension of the dataset. Therefore, the division between the test data is implemented by a set of parallel lines.

We fix the tree number=20, ρ=3 and change the depth within {2, 5, 7, 11} to see the effect of tree depth.

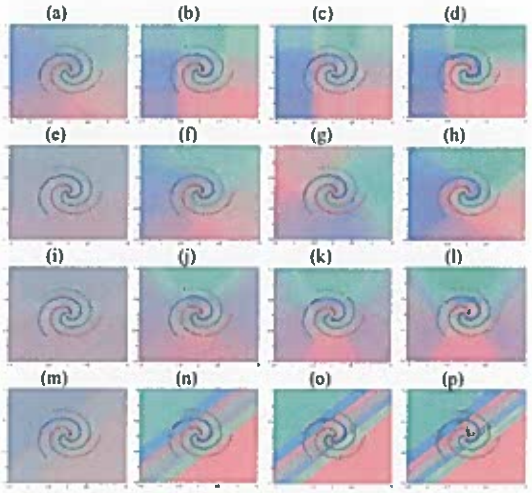


Figure 6 Show the effects of tree depth ranging from {2, 5, 7, 11} : (a), (b), (c) and (d) are the classification of test data using axis-aligned weak learner when tree depth equals 2, 5, 7 and 11 respectively; (e), (f), (g) and (h) are the results of linear weak learner; (i), (j), (k) and (i) are those of non-linear weak leaner; (m), (n), (o) and (p) are those of two-pixel.

Comparing each column in Figure 6, it is clear that when the depth of tree is too low, the difference between colour encoded test data points are not obvious, which means the classification result is bad. It is because when the depth is too low, the number of leaf nodes is small, hence, the ability of the random forest to classify the dataset is relatively low. When the depth reaches 7 and 11, the result of classification becomes better than before for the number of leaves grows with the increase of tree depth, so it is more possible to separate different classes.

To see the effect of randomness parameter, we fix the tree number= 20, the tree depth= 5 and change ρ within {3, 10, 50, 100} and the result is shown as Figure 7.The performance of classification can be improved by adding ρ, however compared with the other two parameters, the effect of ρ is relatively small. The value of ρ controls not only the amount of randomness within each tree but also the amount of correlation between different trees in the forest. When ρ is set to 10 or 50, the transition of the colour is relatively smooth. When ρ=100, the edge is clear, since the decisions of each tree are quite close.

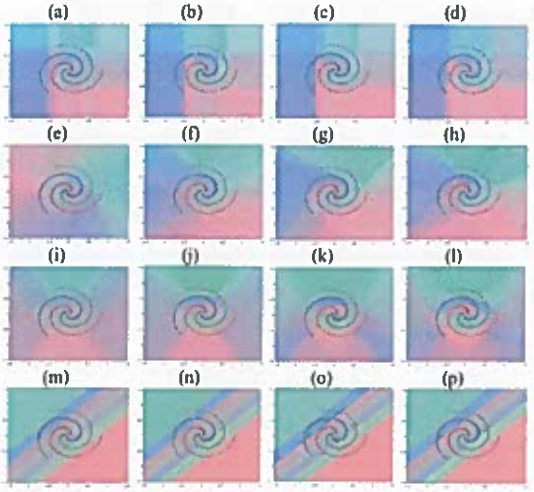


Figure 7 Show the effect of ρ ranging in {3, 10, 50, 100} : (a), (b), (c) and (d) are the classification of test data using axis-aligned weak learner when ρ equals 3, 10, 50 and 100 respectively; (e), (f), (g) and (h) are the results of linear weak learner; (i), (j), (k) and (i) are those of non-linear weak leaner; (m), (n), (o) and (p) are those of two-pixel.

2.3 Experiment with Caltech101 dataset for image categorisation

In this section, RF is applied to classify images from Caltech101 dataset. 10 classes, 15 images per class are randomly selected for training and 15 other images per class are selected for testing.

2.3.1 K-means codebook

K-means clustering is to partition *n* observations into *k* clusters by assigning each observation into the cluster whose center is the nearest one with this observation. In this method, the descriptors are divided into K groups to form a visual codebook.

Here we randomly choose 100k descriptors for K-means clustering to obtain the visual vocabulary with K codewords. By using different value of K, we can build different codebooks. And the vocabulary size in this method is equal to K which is the number of clusters in K-means.

We can derive that the number of histogram bins (K clusters) equals the vocabulary size. When the number of bins changes, it alters the visual vocabulary and results in different histograms with the same image. The vector quantisation process is realized by comparing each visual word with codewords and assigning it to the nearest codeword. (Nearest Neighbor matching). As we can see from Figure 8, if we increase the number of code words, the codewords will be more concentrated in the histogram. During simulation, it is obvious that with the increase of vocabulary size, the time consumption will

Page: 3

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:40:31

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:40:40
good discussions

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:41:01

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:42:00
showing the combinatorial effects of a parameter and different split functions was interesting!

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:40:46

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:16:10

also increase because the vector quantisation process by Nearest Neighbor is time–demanding.

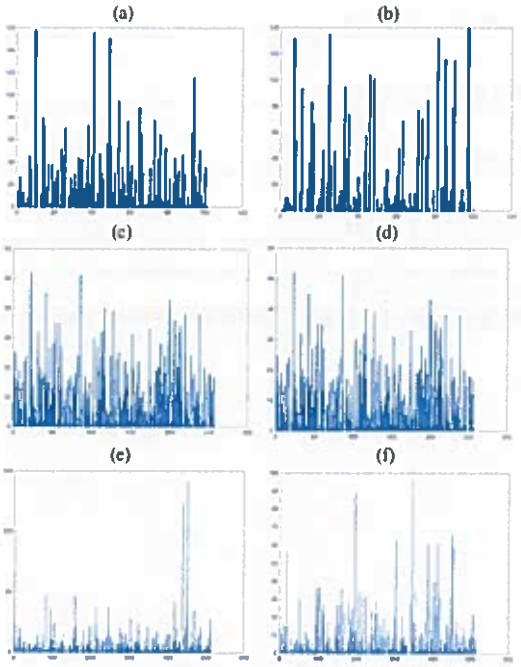
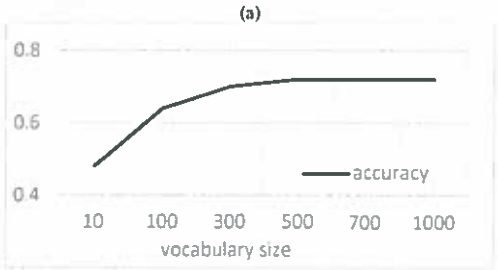


Figure 8 Bag-of-words histograms of one selected training/testing image. (a) and (b) are histograms of four training and testing images respectively with vocabulary size=100; (c) and (d) are those of vocabulary size=256; (e) and (f) are those off vocabulary size=512.

2.3.2 RF classifier

In this section, the training and testing data in the form of bag-of-words obtained by K-means codebook is used to train and test Random Forest. From 2.3.1, we know that the vocabulary size equals K (clusters). By fix the RF parameters (tree number=200, depth=5, ρ=5, axis-aligned weak learner), the impact of the vocabulary size on classification accuracy and computation time is shown in Figure 9.

It is clear from Figure 9 that when vocabulary size is less than 300, with the increase of the vocabulary size, the classification accuracy of RF grows rapidly. However it will spend more time on the construction of visual vocabulary and the vector quantisation process.



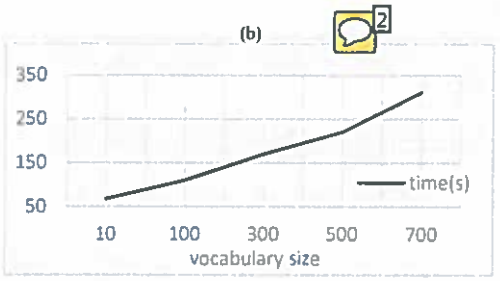


Figure 9 (a) The impact of the vocabulary size on classification accuracy; (b) The impact of the vocabulary size on computation time.

Then we fix the vocabulary size (256) and change the RF parameters to see their impact on recognition accuracy. The results are shown as below.

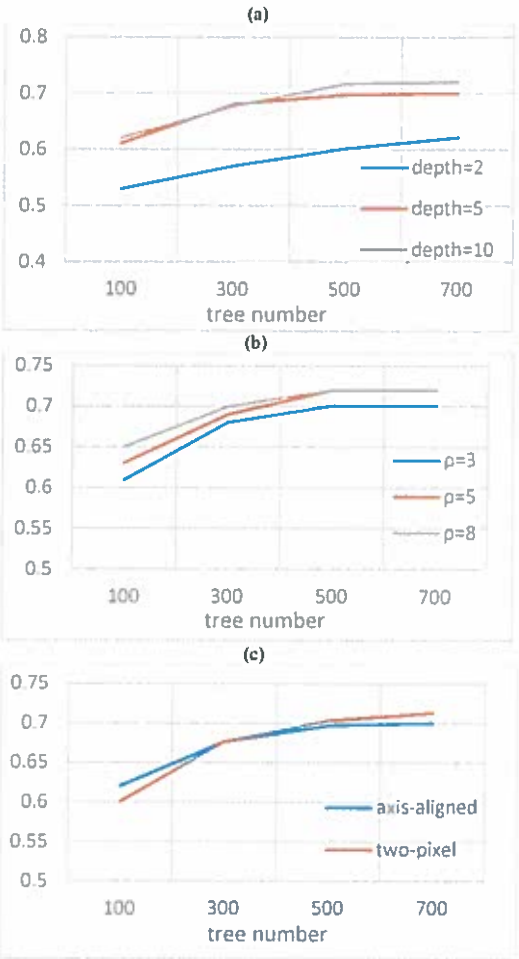


Figure 10 (a) The relationship between classification accuracy and RF depth; (b) The relationship between accuracy and randomness parameter ρ. (c) The accuracy of RF classifier using axis-aligned and two-pixel weak learners.

In Figure 10 (a) and (b), we use axis-aligned weak learner, and vary the number of trees to see the accuracy of RF classifier with different depths and randomness

Page: 4

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:16:33
in terms of accuracy?

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:21:28
very good, is this the time for vector quantisation? or km clustering?

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:18:31
very good

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:17:17

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:17:21
very good

Number: 6 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:19:38
what are pros and cons of bigger vocabulary size? in terms of efficiency? Does a bigger one give us a better accuracy always? (overfitting, noise etc), do we observe any such?

parameter ρ. It is obvious from Figure 10 that before tree number reaches 500, the classification accuracy grows with the increase of tree number. When the tree number is larger than 500, the accuracy tends to be stable. It is because a random forest is an ensemble of randomly trained decision trees which leads to decorrelation between the individual tree predictions and, in turn, results in improved generalization and robustness. So in general, the more trees are generated the better get the results, however, the improvement decreases and the computation complexity increases as the number of trees increases. From (a), when the depth is less than 5, the accuracy increases with the growth of depth. However the depth has little impact on the RF classifier accuracy if the depth reaches 5. If the tree depth is too large, the problem of overfitting will occur for there are too many leaves per tree. From (b), when tree number is under 500, the performance of RF classifier can be improved by adding the randomness parameter for the reason that by adding ρ, the randomness becomes higher and the correlation between different trees gets lower which improves the classification performance. Which directly influences the classification forest's generalization. In (c) the classification accuracy is compared between two RF classifiers using axis-aligned and two-pixel weak learners. Generally, the two classifiers have similar performance.

2.3.3 RF codebook

In this part, the K-means codebook is replaced by RF codebook to generate the visual vocabulary. In this method, the size of the vocabulary equals the number of leaves of the Random Forest. By fixing the RF classifier parameter (tree number=200, depth=5, ρ=5, axis-aligned weak learner) and changing the parameters of RF codebook, the impact of vocabulary size on accuracy and computation time can be obtained. The result is shown as Figure 11.

Compare the result of RF classifier using K-means codebook and RF codebook respectively, it is obvious that, the two lines of accuracy have a similar tendency and in RF codebook classifier, the accuracy has a peak at 142 after which the accuracy tends to be flat. The advantage of RF codebook is its computation time. The computation time of RF codebook is far more less than that of K-means codebook and with the vocabulary size has little impact on computation time. In RF codebook, quantizing the vectors using a learned visual dictionary, i.e. a process that assigns discrete labels to descriptors, with similar descriptors having a high probability of being assigned the same label. During training, RF has a complexity of $O(\sqrt{D}NlogK)$, where N is the number of patches, K is the number of leaf nodes and D denotes the descriptor dimensionality. While in K-means, the complexity equals of $O(DNK)$, where K denotes the number of clusters. In testing, RF is also faster in use: a useful property given that reliable image classification requires large numbers of image patches to be labelled.

Labelling a descriptor with a balanced tree requires $O(logK)$ operations whereas k-means costs $O(KD)$.

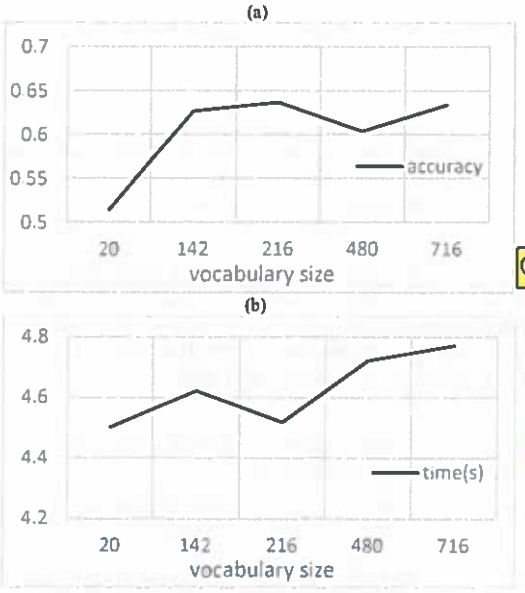
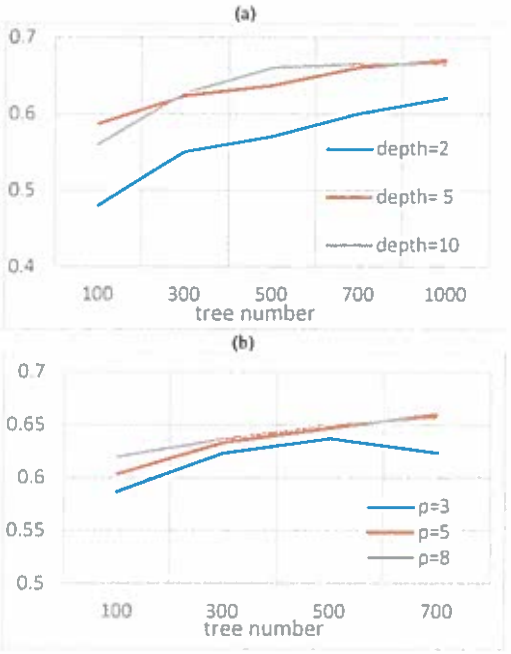


Figure 11 (a) The impact of the vocabulary size on classification accuracy; (b) The impact of the vocabulary size on computation time.

Then we set tree number=15, tree depth=5, randomness parameter ρ=3 and axis-aligned weak learner to generate a RF codebook with the vocabulary size approximately equals 200. After this, we implement the RF classifier using the obtained RF codebook and change parameters in the classifier to see their effects on classification accuracy.



Page: 5

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:24:05
excellent

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:22:30
RF training/testing time measurements?

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:23:02
what do we observe when changing multiple (more than 1) parameters at once? e.g. 3D plot (z: performance, x: parameter 1, y: parameter 2)
do we measure both training time and test time for some different RF parameter values?

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:27:25
do we try different parameters in both RF codebook and RF classifier? what are the best parameters in each? do we observe any difference, if yes, why?
do we obtain any improvements in terms of accuracy, why?
do we measure the vector quantization time for RF codebook, in comparison with k-means? what split function do we use? axis-aligned? or linear??

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:24:49
some success/fail examples, and why?

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:21:06

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:23:48

Number: 8 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:24:18

Number: 9 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:26:43

Number: 10 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:24:59
very good discussions

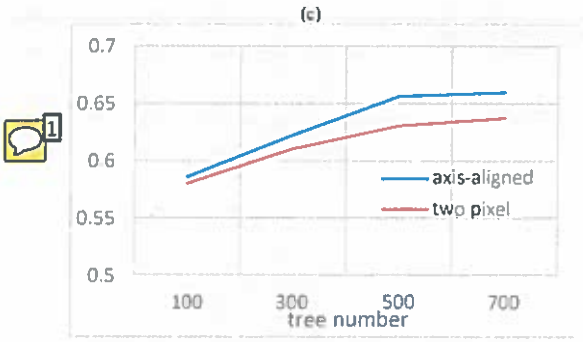


Figure 12 (a) The relationship between classification accuracy and RF depth; (b) The relationship between accuracy and randomness parameter ρ (c) The accuracy of RF classifier using axis-aligned and two-pixel weak learners.

Figure 13 is the confusion matrix of two RF classifiers using K-means and RF codebook respectively. After several trials to change the RF classifier parameters and vocabulary size, the accuracy of RF classifier using K-means codebook reaches 72% and the accuracy of RF codebook is 68%.

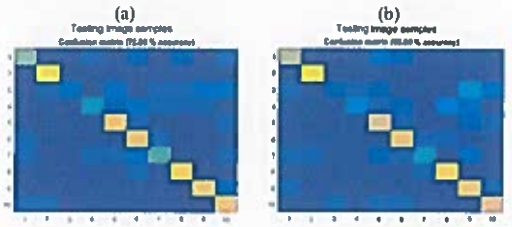


Figure 13 (a) The confusion matrix of RF classifier using K-means codebook (Set tree number=700, depth=10, ρ=5, axis-aligned weak learner and the vocabulary size=256); (b) The confusion matrix of RF classifier using RF codebook (Set tree number=700, depth=5, ρ=5, axis-aligned weak learner and the vocabulary size=202)

III CONCLUSION

In this coursework, we observe different parameters' impact on decision forest. The depth of a decision tree affects the classification accuracy, because by adding depth, more leaves of a tree is generated which leads to high confidence in prediction. However, as the depth increases, the complexity of model grows and there is no principle way to tree pruning .Thus the model overfits the data resulting in the increment of real loss. While the random forest is an ensemble of randomly trained decision trees. By adding the random parameter and tree number, the random forest tends to be hardly overfitting and, apart from this the classification accuracy will also increase.

Bag of visual vocabulary (codebook) is a way to construct a feature vector for classification. Visual coding based on K-means vector quantization is effective but slow because it relies on Nearest Neighbor search, which remains hard to accelerate in high dimensional descriptor spaces. It is also computationally expensive owing to the cost of assigning visual descriptors to visual codewords during training and testing. The advantage of adopting random forest for codebook classification and vector quantisation is its computation speed. Apart from this, classification trees contain a lot of valuable discriminative information in descriptor space that helps the subsequent classification problem for object categorisation. While in this experiment, we didn't achieve the desired result, and the optimum accuracy of classifier using K-means codebook and RF codebook is 72% and 68% respectively.

IV REFERENCE

[1] Tae-Kyun Kim. "Machine Learning for Computer Vision" Lecture notes, Department of Electrical and Electronic Engineering, Imperial College London, UK

[2] Kalal Z, Matas J, Mikolajczyk K, 2010, P-N Learning: Bootstrapping Binary Classifiers by Structural Constraints, 23rd IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Publisher: IEEE COMPUTER SOC, Pages: 49-56, ISSN: 1063-6919.

[3] Cai H, Yan F, Mikolajczyk K, 2010, Learning Weights for Codebook in Image Classification and Retrieval, IEEE Conference on Computer Vision and Pattern Recognition, Publisher: IEEE, Pages: 2320-2327, ISSN: 1063-6919

[4] Tong, W.; Hong, H.; Fang, H.; Xie, Q.; Perkins, R. Decision forest: combining the predictions of multiple independent decision tree models. J. Chem. Inf. Comput. Sci. 2003, 43, 525-531

[5] Liaw, A.; Wiener, M. Classification and regression by randomForest. R News 2002, 2/3, 18-22.

[6] Freund, Y.; Schapire, R. Experiments with a new Boosting algorithm. Proc. 13th Int. Conf. Mach. Learn. 1996, 148-156.

Page: 6

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:25:15
excellent

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:26:57

V APPENDIX

5.1 Codes for K-means codebook in getData.m:

```
% K-means clustering
        numBins = 256; % for
instance,
        X=desc_sel;
%
X=single(cat(2,desc_tr{:}));

        [C,idx] =
vl_kmeans(X,numBins);
        C=double(C);
        Hist = zeros(1,numBins);
        for i=1:numBins
            Hist(i)=sum(idx==i);
        end
%         figure;
%         bar(Hist);
%         xlabel('Codewords')
%         ylabel('frequency');
%         title('Bag of wards');

        disp('Encoding Images...')
        % Vector Quantisation

        % write your own codes here
        % ...
        Y=[];
        for i=1:length(classList)

Y=[Y;i*ones(length(imgIdx_tr),1)];
        end


X=zeros(length(classList)*length(imgI
dx_tr),numBins);
        for i = 1:length(classList)
            for j=1:length(imgIdx_tr)
                for
m=1:size(desc_tr{i,j},2)
                    DD=inf;
                    for n=1:numBins

dd=norm(double(desc_tr{i,j}(:,m))-
C(:,n));
                        if dd<DD
                            DD=dd;
                            pin=n;
                        end
                    end
                    X((i-
1)*15+j,pin)=X((i-1)*15+j,pin)+1;
                end
            end
        end

        data_train=[X Y];
% Quantisation
        [m,n]=size(desc_te);
        X=zeros(m*n,numBins);
        for i=1:m
            for j=1:n
                for
k=1:size(desc_te{i,j},2)
                    DD=inf;
                    for nn=1:numBins

dd=norm(double(desc_te{i,j}(:,k))-
C(:,nn));
                        if dd<DD
                            DD=dd;
                            pin=nn;
                        end
                    end
                    X((i-
1)*15+j,pin)=X((i-1)*15+j,pin)+1;

                end
            end
        end
%         data_query=[X
zeros(m*n,1)];
        data_query=[X Y];
```

5.2 Codes for RF codebook in getData.m:

```
% Build visual vocabulary (codebook)
for 'Bag-of-Words method'
        desc_tr=desc_tr.';
        count=[];
        [desc_sel,Y] =
vl_colsubset(cat(2,desc_tr{:}),
10e4); % Randomly select 100k SIFT
descriptors for clustering
        for ii=1:1:10
            for jj=1:1:15
                [mm
nn]=size(desc_tr{jj,ii});
                count=[count nn];
            end
        end


idx=zeros(1,size(desc_sel,2));
        desc_sel=double(desc_sel.');
        idx=idx.';
        data=[desc_sel,idx];
        % rf
        param.num = 10;          %
Number of trees
        param.depth = 5;         %
trees depth
        param.splitNum = 3;      %
Number of split functions to try
        param.split = 'IG';      %
Currently support 'information gain'
only
        param.splitfunction=1;
```

```
        trees =
growTrees(data,param);
        NN=[];
          for ii=1:1:10
            for jj=1:1:15
                leaves =
testTrees_fast(desc_tr(jj,ii).',trees
);
                leaves(~leaves)=1;
                [r
c]=size(trees(1).prob);
                binnums=[1:r];
                N=[];

N=histc(leaves(:),binnums);
                NN=[NN,N];
            end
          end
         NN=NN.';
        ref=[];
        for nn=1:1:10
            ref1=nn*ones(15,1);
            ref=[ref,ref1];
        end
        ref=ref(:);
        data_train=[NN,ref];
% Quantisation
  desc_te=desc_te.';
        NN_test=[];
           for ii=1:1:10
            for jj=1:1:15
                leaves_test =
testTrees_fast(desc_te(jj,ii).',trees
);

leaves_test(~leaves_test)=1;
                binnums=[1:r];
                N_test=[];

N_test=histc(leaves_test(:),binnums);

NN_test=[NN_test,N_test];
            end
          end
         NN_test=NN_test.';
        data_query=[NN_test,ref];
```